

Aula 1: Introdução — redes neurais como aproximadores universais

Conexionismo • Perceptron • Universalidade • O papel da profundidade

Prof. Marcelo Keese Albertini

GBC073 — Inteligência Computacional • Universidade Federal de Uberlândia
2026



PARTE 1

Por que esta disciplina, agora



O que “inteligência computacional” significa em 2026

A ementa lista três paradigmas; a história já votou entre eles

- ▶ **Redes neurais artificiais** — de perceptrons a *Transformers*: hoje são a tecnologia dominante em visão, linguagem, fala, biologia e jogos

O que “inteligência computacional” significa em 2026

A ementa lista três paradigmas; a história já votou entre eles

- ▶ **Redes neurais artificiais** — de perceptrons a *Transformers*: hoje são a tecnologia dominante em visão, linguagem, fala, biologia e jogos
- ▶ **Computação evolutiva** — otimização *sem gradiente*: veremos onde ainda é competitiva (e por que perdeu o centro do palco)

O que “inteligência computacional” significa em 2026

A ementa lista três paradigmas; a história já votou entre eles

- ▶ **Redes neurais artificiais** — de perceptrons a *Transformers*: hoje são a tecnologia dominante em visão, linguagem, fala, biologia e jogos
- ▶ **Computação evolutiva** — otimização *sem gradiente*: veremos onde ainda é competitiva (e por que perdeu o centro do palco)
- ▶ **Sistemas nebulosos** — lógica com graus de verdade: a semente histórica das *relaxações diferenciáveis* que o deep learning usa em todo lugar

O que “inteligência computacional” significa em 2026

A ementa lista três paradigmas; a história já votou entre eles

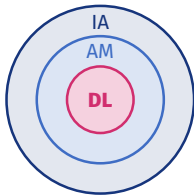
- ▶ **Redes neurais artificiais** — de perceptrons a *Transformers*: hoje são a tecnologia dominante em visão, linguagem, fala, biologia e jogos
- ▶ **Computação evolutiva** — otimização *sem gradiente*: veremos onde ainda é competitiva (e por que perdeu o centro do palco)
- ▶ **Sistemas nebulosos** — lógica com graus de verdade: a semente histórica das *relaxações diferenciáveis* que o deep learning usa em todo lugar

Intuição

O fio condutor do semestre: **tudo que é diferenciável se treina por gradiente** — e quase tudo que importava foi tornado diferenciável. Os outros paradigmas viram contraponto: o que fazer quando o gradiente *não* existe.

Três círculos concêntricos

Onde o deep learning mora — e o que cada fronteira acrescenta



- ▶ **Inteligência artificial** — qualquer técnica que faça o computador imitar comportamento humano
- ▶ **Aprendizado de máquina** — a habilidade de aprender sem ser explicitamente programado
- ▶ **Aprendizado profundo** — extrair padrões dos dados brutos com redes de múltiplas camadas

Intuição

A frase-resumo: ensinar o computador a aprender a tarefa a partir de dados brutos — em vez de receber a regra pronta.

O momento em que estamos

Este não é um curso de história — mas a história acabou de nos dar razão

- ▶ **2024:** Nobel de *Física* para Hopfield e Hinton (redes neurais); de *Química* para o AlphaFold
- ▶ **Modelos de linguagem** escrevem código, provam teoremas e passam em exames profissionais
- ▶ **Difusão** gera imagens, áudio, vídeo e moléculas
- ▶ Tudo é, no fundo, **uma única receita**: redes profundas + gradiente + dados + computação

Cuidado

A receita é simples de enunciar e difícil de dominar. Este curso existe para você entender *cada peça* — a ponto de implementá-la do zero.

```
foto: imagens/hinton.jpg
rode imagens/baixar-imagens.sh
```

G. Hinton — Nobel de Física 2024, com J. Hopfield

Foto: Eviatar Bach, Wikimedia Commons, CC BY-SA

Por que só agora?

As ideias centrais são dos anos 1950–1980; o que mudou foi a escala

1. Dados

Conjuntos maiores; coleta e armazenamento baratos.

2. Hardware

GPUs: a operação central — multiplicação de matrizes — é massivamente paralelizável.

3. Software

Técnicas melhores, modelos novos, bibliotecas maduras.

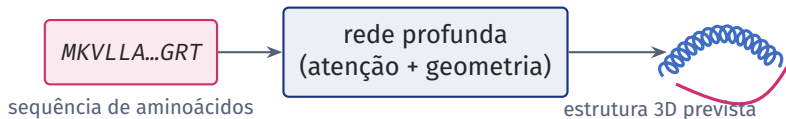
1952 gradiente descendente estocástico • **1958** perceptron com pesos aprendidos • **1986** retropropagação • **1995** redes convolucionais profundas

Cuidado

As ideias centrais desta aula são dos anos 1950–1980. O que mudou foi a escala — e escala, aqui, muda o que é possível, não apenas o quão rápido.

O que uma rede profunda fez pela biologia

AlphaFold: o problema de 50 anos da estrutura de proteínas



- ▶ Em 2020 (CASP14), atingiu precisão comparável à experimental — problema aberto desde os anos 1970
- ▶ Banco público com centenas de milhões de proteínas (CC BY 4.0); Nobel de *Química* 2024 para Hassabis e Jumper

Intuição

Por dentro é a receita desta disciplina: modelo diferenciável de ponta a ponta, treinado por gradiente — feito dos blocos de *atenção* em que este curso termina.

Em aula

- ▶ Teoria *com* código: toda ideia central aparece em PyTorch, projetado e lido junto
- ▶ Perguntas para discutir — pense antes do clique da resposta
- ▶ Simuladores interativos para levar para casa

Fora de aula

- ▶ Leituras: Goodfellow et al. (cap. 1, 6), *d2l.ai*, Nielsen
- ▶ Projeto: reprodução de um artigo, em grupos, com código rodando de verdade

- ▶ Material-base: CMU 11-785 (S26), MIT 6.S191, Stanford CS231n — adaptados e traduzidos para a nossa ementa

PARTE 2

Conexionismo: a aposta que demorou a dar certo



IA simbólica

- ▶ Inteligência = manipular *símbolos* com regras
- ▶ Conhecimento escrito à mão por especialistas
- ▶ Frágil fora do previsto

Conexionismo

- ▶ Inteligência *emerge* de unidades simples conectadas
- ▶ Conhecimento fica nos **pesos** — *aprendido* de dados
- ▶ Inspiração: o cérebro ($\sim 10^{11}$ neurônios, $\sim 10^{14}$ sinapses)

Intuição

A pergunta da aula de hoje: unidades tão simples — somar e comparar com um limiar — *conseguem* computar algo interessante? Sim, e o argumento é surpreendentemente concreto.

- ▶ **1943** McCulloch & Pitts: o neurônio como *unidade de limiar* — redes de neurônios computam lógica
- ▶ **1949** Hebb: sinapses que se reforçam com o uso — aprendizado local, sem professor

- ▶ **1943** McCulloch & Pitts: o neurônio como *unidade de limiar* — redes de neurônios computam lógica
- ▶ **1949** Hebb: sinapses que se reforçam com o uso — aprendizado local, sem professor
- ▶ **1958** Rosenblatt: o **Perceptron** — pela primeira vez, os pesos são *aprendidos* de exemplos
- ▶ **1969** Minsky & Papert: limites do perceptron simples (XOR) $\Rightarrow$ primeiro “inverno”

- ▶ **1943** McCulloch & Pitts: o neurônio como *unidade de limiar* — redes de neurônios computam lógica
- ▶ **1949** Hebb: sinapses que se reforçam com o uso — aprendizado local, sem professor
- ▶ **1958** Rosenblatt: o **Perceptron** — pela primeira vez, os pesos são *aprendidos* de exemplos
- ▶ **1969** Minsky & Papert: limites do perceptron simples (XOR) $\Rightarrow$ primeiro “inverno”
- ▶ **1986** Rumelhart, Hinton & Williams: retropropagação populariza o treino de redes *multicamadas*
- ▶ **2012** AlexNet: GPUs + dados — deep learning vence em visão e nunca mais sai

- ▶ **1943** McCulloch & Pitts: o neurônio como *unidade de limiar* — redes de neurônios computam lógica
- ▶ **1949** Hebb: sinapses que se reforçam com o uso — aprendizado local, sem professor
- ▶ **1958** Rosenblatt: o **Perceptron** — pela primeira vez, os pesos são *aprendidos* de exemplos
- ▶ **1969** Minsky & Papert: limites do perceptron simples (XOR) $\Rightarrow$ primeiro “inverno”
- ▶ **1986** Rumelhart, Hinton & Williams: retropropagação populariza o treino de redes *multicamadas*
- ▶ **2012** AlexNet: GPUs + dados — deep learning vence em visão e nunca mais sai
- ▶ **2017–hoje** o Transformer (*atenção*) unifica linguagem, visão e áudio — é onde este curso quer chegar

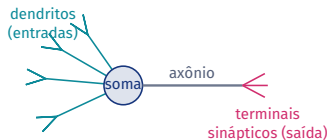
PARTE 3

0 perceptron por dentro



O original biológico

O neurônio de verdade — e o mapa que McCulloch e Pitts fizeram dele



Esquema; o neurônio real tem 10^3 – 10^4 sinapses de entrada.

- ▶ Dendritos → **entradas** x_i
- ▶ Eficácia da sinapse → **peso** w_i
- ▶ Soma integra → $w^\top x$
- ▶ Limiar de disparo → **ativação** φ

Intuição

A abstração de 1943 mantém só somar e comparar com limiar — e ainda assim basta para computar qualquer coisa.

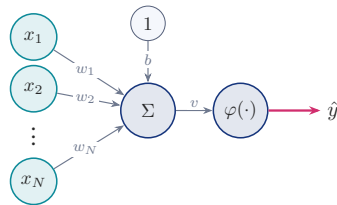
Definição — Neurônio de limiar (perceptron)

Campo local induzido $v = \mathbf{w}^\top \mathbf{x} + b$; saída $\hat{y} = \varphi(v)$, com φ o degrau: $\varphi(v) = 1$ se $v \geq 0$, senão 0.

- ▶ w : quanto cada entrada *importa*
- ▶ b : quão fácil é disparar
- ▶ φ : a decisão (não linearidade)

Intuição

Geometria: $\mathbf{w}^\top \mathbf{x} + b = 0$ é um *hiperplano*; w é a normal, b desloca. O perceptron é um separador linear — e essa frase inteira já contém a sua limitação.



Do vetor ao lote: a forma matricial

A mesma conta, de uma entrada por vez a um lote inteiro

$$\hat{y} = \varphi(w_0 + \mathbf{x}^\top \mathbf{w}) \quad \implies \quad Z = XW + b, \quad A = \varphi(Z)$$

- ▶ O viés w_0 desloca a fronteira de decisão — sem ele, ela é obrigada a passar pela origem
- ▶ Com um lote de B exemplos, a entrada vira uma matriz: X é $B \times m$, W é $m \times 1$ e a saída, $B \times 1$ — o lote atravessa a conta intocado
- ▶ Tudo o que um perceptron faz é um produto interno seguido de uma função escalar — exatamente a operação que as GPUs fazem bem

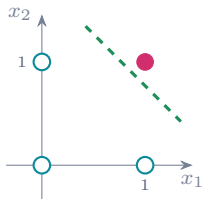
Intuição

Uma rede profunda é, em essência, uma pilha de multiplicações de matrizes intercaladas com não linearidades. Erros de dimensão são a fonte número um de bugs: as colunas da entrada casam com as linhas dos pesos — sempre.

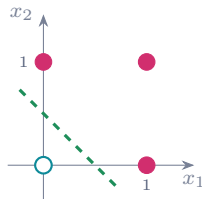
A geometria da decisão

Quatro pontos bastam para ver o que o perceptron pode — e o que não pode

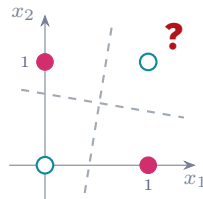
E: separável



OU: separável



XOR: impossível



- ▶ ● classe 1 ○ classe 0 — uma reta $w^T x + b = 0$ decide tudo
- ▶ No XOR, qualquer reta deixa um ponto do lado errado: o problema não é o algoritmo de treino, é a **família de funções**

Perceptrons são portas lógicas

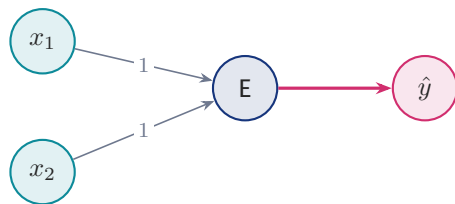
O argumento de McCulloch–Pitts, em três linhas de pesos

Com entradas booleanas $x_i \in \{0, 1\}$ e degrau em $v \geq 0$:

- ▶ **E:** $w = (1, 1), b = -1,5 \Rightarrow$ dispara só com $x_1 = x_2 = 1$
- ▶ **OU:** $w = (1, 1), b = -0,5$
- ▶ **NÃO:** $w = -1, b = 0,5$

Teorema — Universalidade booleana

$\{E, OU, NÃO\}$ é um conjunto universal. Logo, redes de perceptrons computam **qualquer função booleana** — qualquer circuito digital, qualquer tabela-verdade.



O neurônio é um *votante com limiar*; a lógica é o caso particular em que os votos são inteiros.

Algoritmo — Regra do perceptron (1958)

Para cada exemplo (x, y) com $y \in \{0, 1\}$: calcule $\hat{y} = \varphi(\mathbf{w}^\top \mathbf{x} + b)$ e, se errou, corrija na direção do erro:

$$\mathbf{w} \leftarrow \mathbf{w} + \underbrace{\eta}_{\text{taxa de aprendizado}} \underbrace{(y - \hat{y})}_{\text{erro}} \mathbf{x} \quad b \leftarrow b + \eta (y - \hat{y})$$

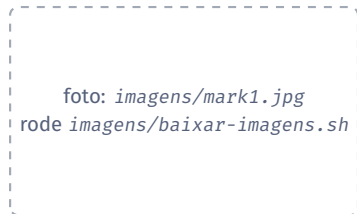
Teorema — Convergência do perceptron (Novikoff, 1962)

Se os dados são **linearmente separáveis**, a regra converge em um número finito de atualizações. Se não são — ela oscila para sempre.

- Note a forma $\mathbf{w} \leftarrow \mathbf{w} + \Delta \mathbf{w}$: é o *ancestral direto* da descida de gradiente que usaremos o semestre todo

1958, em hardware: o Mark I Perceptron

Antes de ser uma linha de PyTorch, o peso era um botão girado por um motor



O Mark I Perceptron, do Cornell Aeronautical Laboratory

Foto: National Museum of the U.S. Navy — domínio público

- ▶ “Retina” de 20×20 fotocélulas: x com 400 dimensões
- ▶ Cada peso w_i era um **potenciômetro**; a regra girava os botões com motores elétricos
- ▶ Em 1958, a imprensa prometeu máquinas que andariam e falaria

Intuição

Hype seguido de decepção não é invenção da nossa década — o ciclo 1958–1969 calibra bem a leitura das manchetes de hoje.

A regra do perceptron em PyTorch

Dez linhas — aprendendo a porta E

```
import torch

X = torch.tensor([[0.,0.],[0.,1.],[1.,0.],[1.,1.]])
y = torch.tensor([0., 0., 0., 1.])          # porta E

w, b = torch.zeros(2), torch.zeros(1)
for epoca in range(10):
    for xi, yi in zip(X, y):
        y_hat = (w @ xi + b >= 0).float()    # degrau
        erro = yi - y_hat                    # -1, 0 ou +1
        w += erro * xi                       # regra do perceptron
        b += erro
```

- ▶ Sem `backward()`, sem gradiente: a regra é *ad hoc* — funciona porque o degrau + erro discreto conspiram a favor

Para discutir em aula

O perceptron simples aprende E e OU sem dificuldade. Então, com paciência e uma taxa de aprendizado bem escolhida, ele também acaba aprendendo XOR ($x_1 \oplus x_2$).

Verdadeiro / Falso

Para discutir em aula

O perceptron simples aprende E e OU sem dificuldade. Então, com paciência e uma taxa de aprendizado bem escolhida, ele também acaba aprendendo XOR ($x_1 \oplus x_2$).

Verdadeiro / Falso

Fechamento

Falso — e não é questão de paciência. Vimos a geometria: **nenhuma reta** separa $\{(0, 0), (1, 1)\}$ de $\{(0, 1), (1, 0)\}$, e o teorema de convergência exigia exatamente separabilidade. Foi este exemplo, em Minsky & Papert (1969), que congelou a área por uma década. A saída não é um perceptron melhor: é **empilhar** perceptrons.

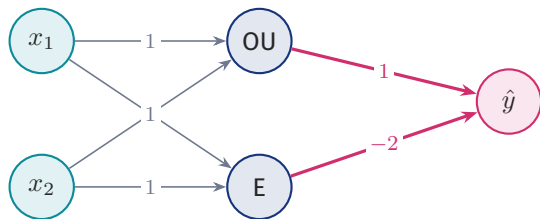
PARTE 4

Profundidade: de portas a aproximadores universais



Uma camada oculta resolve o XOR

XOR = (OU) E NÃO (E) — a rede escreve essa fórmula em pesos



Limiares: h_1 dispara com $v \geq 0,5$; h_2 com $v \geq 1,5$; $\hat{y}$ com $v \geq 0,5$. Verifique nas quatro entradas.

- ▶ h_1 e h_2 são *features* construídas: “algum ligado” e “ambos ligados”
- ▶ Na saída, uma única reta sobre (h_1, h_2) resolve — o próximo slide mostra isso acontecendo

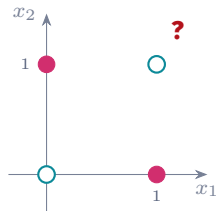
Intuição

A tese do deep learning inteiro: camadas não adicionam só capacidade — adicionam **representações**.

O que a camada oculta faz com o espaço

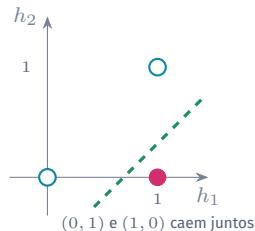
Os mesmos quatro pontos, antes e depois de (h_1, h_2)

Espaço de entrada (x_1, x_2)



$\xrightarrow{h}$
camada oculta

Espaço oculto (h_1, h_2)



- ▶ A camada oculta **dobra o espaço**: dois pontos que precisavam da mesma resposta são mapeados para o *mesmo lugar*
- ▶ Depois disso, o problema volta a ser linear — e o perceptron de saída dá conta

Funções de ativação comuns

Três suavizações do degrau — cada uma com sua derivada

Sigmoide

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Tangente hiperbólica

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$\tanh'(z) = 1 - \tanh^2(z)$$

ReLU

$$\text{ReLU}(z) = \max(0, z)$$

$\text{ReLU}'(z)$: 1 se $z > 0$; 0 caso contrário

- ▶ Todas são **não lineares** — e é exatamente esse o ponto
- ▶ Nas camadas ocultas de hoje, **ReLU** é o padrão; na saída, a ativação depende da tarefa

Para discutir em aula

Uma rede de dez camadas com ativação identidade $g(z) = z$ tem o mesmo poder expressivo de uma única camada linear, não importa quantos neurônios cada camada tenha.

Verdadeiro / Falso

Para discutir em aula

Uma rede de dez camadas com ativação identidade $g(z) = z$ tem o mesmo poder expressivo de uma única camada linear, não importa quantos neurônios cada camada tenha.

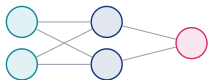
Verdadeiro / Falso

Fechamento

Verdadeiro. Se g é linear, a composição de camadas volta a ser uma função linear: $y = W_2(W_1\mathbf{x} + b_1) + b_2 = (W_2W_1)\mathbf{x} + (W_2b_1 + b_2)$. Uma rede de 100 camadas teria o poder de uma só. A não linearidade é o que permite a cada camada acrescentar **representação de verdade**.

Propagação direta, passo a passo

Rede 2→2→1, ReLU na camada oculta, sigmoide na saída — um exemplo numérico completo



Entrada $x = (1, 2)$; pesos e vies:

$$W^{(1)} = \begin{pmatrix} 0,5 & -1,0 \\ 1,5 & 0,5 \end{pmatrix}, b^{(1)} = (0, -1)$$

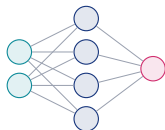
$$W^{(2)} = \begin{pmatrix} 1,0 \\ 2,0 \end{pmatrix}, b^{(2)} = -2$$

Passo	Operação	Resultado
1	$z^{(1)} = xW^{(1)} + b^{(1)}$	$(3,5, -1,0)$
2	$a^{(1)} = \text{ReLU}(z^{(1)})$	$(3,5, 0)$
3	$z^{(2)} = a^{(1)}W^{(2)} + b^{(2)}$	$1,5$
4	$\hat{y} = \sigma(z^{(2)})$	$0,8176$

Confira: $3,5 = 1(0,5) + 2(1,5) + 0$ e $\hat{y} = 1/(1 + e^{-1,5})$.

Cuidado

Repare no passo 2: a ReLU zerou a segunda unidade oculta. Para esta entrada ela não contribui em nada — e, na retropropagação, não receberá gradiente. É assim que a ReLU produz **ativações esparsas**.



Troque o degrau por uma φ suave (σ , $\tanh$, ReLU) e tudo vira diferenciável. Intuição do teorema: cada neurônio contribui um “degrau”; a soma de degraus suficientes vira uma tabela de consulta suave.

Teorema — Aproximação universal (Cybenko 1989; Hornik 1991)

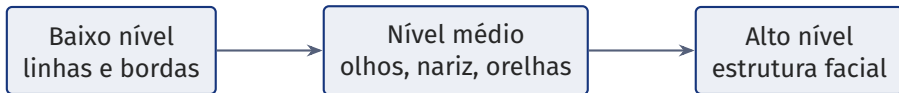
Uma rede com *uma* camada oculta e ativação não polinomial aproxima qualquer função contínua em um compacto de $\mathbb{R}^n$.

Cuidado

O teorema diz que os pesos *existem* — não quantos neurônios custam, nem como *encontrá-los*. Representação, não aprendizado.

A hierarquia que emerge sozinha

Ninguém programou “detecte uma borda” — a rede organizou-se assim durante o treino



- ▶ Em reconhecimento facial, cada camada aprende um nível de abstração
- ▶ A hierarquia *emerge* do treinamento — é isso que a palavra “profundo” compra

Intuição

É a versão grande do XOR: lá, a camada oculta construiu “algum ligado” e “ambos ligados”; aqui, constrói bordas, partes e rostos. O mecanismo é o mesmo — muda a escala.

Por que fundo, e não largo?

O resultado mais importante da aula — e o mais fácil de perder

Paridade de n bits, $x_1 \oplus \dots \oplus x_n$:

- ▶ Com **uma** camada oculta, a construção por tabela-verdade usa 2^{n-1} neurônios; circuitos rasos de E/OU exigem tamanho exponencial (Håstad, 1986)

Por que fundo, e não largo?

O resultado mais importante da aula — e o mais fácil de perder

Paridade de n bits, $x_1 \oplus \dots \oplus x_n$:

- ▶ Com **uma** camada oculta, a construção por tabela-verdade usa 2^{n-1} neurônios; circuitos rasos de E/OU exigem tamanho exponencial (Håstad, 1986)
- ▶ Com **profundidade**: uma árvore de XORs de 2 entradas usa $\mathcal{O}(n)$ neurônios em $\mathcal{O}(\log n)$ camadas

Por que fundo, e não largo?

O resultado mais importante da aula — e o mais fácil de perder

Paridade de n bits, $x_1 \oplus \dots \oplus x_n$:

- ▶ Com **uma** camada oculta, a construção por tabela-verdade usa 2^{n-1} neurônios; circuitos rasos de E/OU exigem tamanho exponencial (Håstad, 1986)
- ▶ Com **profundidade**: uma árvore de XORs de 2 entradas usa $\mathcal{O}(n)$ neurônios em $\mathcal{O}(\log n)$ camadas

Intuição

Profundidade compra **reuso de computação intermediária**: cada camada calcula subexpressões que as seguintes combinam; a rede rasa trata caso a caso. É por isso que você escreve programas com funções, não uma tabela.

Por que fundo, e não largo?

O resultado mais importante da aula — e o mais fácil de perder

Paridade de n bits, $x_1 \oplus \dots \oplus x_n$:

- ▶ Com **uma** camada oculta, a construção por tabela-verdade usa 2^{n-1} neurônios; circuitos rasos de E/OU exigem tamanho exponencial (Håstad, 1986)
- ▶ Com **profundidade**: uma árvore de XORs de 2 entradas usa $\mathcal{O}(n)$ neurônios em $\mathcal{O}(\log n)$ camadas

Intuição

Profundidade compra **reuso de computação intermediária**: cada camada calcula subexpressões que as seguintes combinam; a rede rasa trata caso a caso. É por isso que você escreve programas com funções, não uma tabela.

- ▶ **Fontes**: Raj, CMU 11-785, Lecs. 1–2; Håstad (1986); para redes contínuas, Telgarsky (2016) e Eldan & Shamir (2016) — o “deep” é contagem de circuito, não moda

Para discutir em aula

Pelo teorema da aproximação universal, uma camada oculta basta. Logo, redes profundas são um luxo de engenharia: qualquer coisa que uma rede profunda faz, uma rede rasa faz igualmente bem na prática.

Verdadeiro / Falso

Para discutir em aula

Pelo teorema da aproximação universal, uma camada oculta basta. Logo, redes profundas são um luxo de engenharia: qualquer coisa que uma rede profunda faz, uma rede rasa faz igualmente bem na prática.

Verdadeiro / Falso

Fechamento

Falso. “Existe uma rede rasa equivalente” é verdade; “do mesmo tamanho” não é. Para famílias como a paridade, a versão rasa é **exponencialmente** maior — impossível de armazenar, quanto mais de treinar. Universalidade é sobre *o que é representável*; profundidade é sobre *a que custo*. O semestre vive nessa distância.

PARTE 5

O que vem pela frente



A ativação de saída define a tarefa

Nas camadas ocultas, ReLU; na última camada, a escolha importa

Tarefa	Ativação de saída	Interpretação
Regressão	nenhuma (linear)	valor real, sem restrição
Classificação binária	sigmoide, 1 saída	$P(y = 1 \mathbf{x})$
Classificação multiclasse	softmax, K saídas	distribuição sobre K classes
Multirrótulo	sigmoide, K saídas	K probabilidades independentes

Definição — Softmax

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad \Rightarrow \quad \sum_{i=1}^K \text{softmax}(\mathbf{z})_i = 1$$

Intuição

A sigmoide age em cada saída isoladamente; o softmax *acopla* as saídas — subir uma classe baixa as outras.

Prévia: o XOR treinado por gradiente

O que a regra do perceptron não conseguiu, o gradiente resolve em 15 linhas

```
import torch, torch.nn as nn

X = torch.tensor([[0.,0.],[0.,1.],[1.,0.],[1.,1.]])
y = torch.tensor([[0.],[1.],[1.],[0.]]) # XOR

modelo = nn.Sequential(nn.Linear(2, 2), nn.Tanh(),
                       nn.Linear(2, 1), nn.Sigmoid())
otim = torch.optim.SGD(modelo.parameters(), lr=0.5)
perda = nn.BCELoss()

for passo in range(5000):
    otim.zero_grad()
    L = perda(modelo(X), y) # quao errado estou?
    L.backward() # retropropagacao
    otim.step()
```

A perda: medindo o erro

O que a *BCELoss* da prévia faz — e por que ela, e não o erro quadrático

Definição — Perda empírica $J(\mathbf{w})$

$J(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n L(f(\mathbf{x}^{(i)}; \mathbf{w}), y^{(i)})$ — função objetivo, custo ou risco empírico; treinar é procurar $\arg \min_{\mathbf{w}} J(\mathbf{w})$.

Entropia cruzada binária

Classificação: a saída é uma probabilidade.

$$J = -\frac{1}{n} \sum_i [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

Erro quadrático médio

Regressão: a saída é um real contínuo.

$$J = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2$$

Cuidado

EQM + sigmoide carrega $\sigma'(z)$ no gradiente, que some quando o modelo está confiante e errado; a BCE cancela o σ' e sobra o erro $y - \hat{y}$.

`L.backward()` $\implies \frac{\partial E}{\partial w}$ para todos os pesos, pela regra da cadeia

$$\text{otim.step()} \implies w \leftarrow w - \eta \frac{\partial E}{\partial w}$$

- ▶ Compare com a regra do perceptron: mesma forma $w \leftarrow w + \Delta w$, mas agora Δw vem de um **princípio** — minimizar uma perda diferenciável — e não de uma heurística
- ▶ Isso escala do nosso XOR de 9 parâmetros aos modelos de linguagem com 10^{12} : *o algoritmo é o mesmo*
- ▶ Aulas 2–4: descida de gradiente, grafo computacional e a retropropagação implementada do zero (nosso mini-autograd)

Redes neurais e deep learning

Perceptron → MLP → retropropagação e autograd → otimização e regularização modernas → CNNs → sequências → Hopfield moderno → atenção e Transformers

Computação evolutiva

Otimização *sem* gradiente: AGs, estratégias evolutivas, CMA-ES. Quando a derivada não existe — e o custo de viver sem ela.

Sistemas nebulosos

Pertinência gradual → relaxações diferenciáveis: softmax, *gating*, atenção como consulta “fuzzy”.

- ▶ O bloco **neural** carrega a maior parte da carga horária; os outros dois entram como *contraponto*
- ▶ Ao final, você deve saber ler um artigo atual de deep learning — e reproduzi-lo

Os paradigmas de aprendizado

A mesma rede e o mesmo gradiente — o que muda é de onde vem o sinal

Paradigma	Sinal de aprendizado	Exemplo de uso
Supervisionado	rótulo correto por exemplo	diagnóstico por imagem
Não supervisionado	nenhum rótulo	segmentação de clientes
Auto-supervisionado	rótulo derivado do próprio dado	pré-treino de modelos de linguagem
Por reforço	recompensa escalar, possivelmente atrasada	jogos, robótica

Intuição

Nas quatro linhas da tabela, a rede é a mesma e o gradiente é calculado por retropropagação. Um modelo de linguagem moderno usa três delas em sequência: pré-treino auto-supervisionado, ajuste supervisionado e alinhamento por reforço (tema da disciplina GBC063).

Simuladores interativos desta unidade

Para mexer em casa — cada um cabe numa página HTML, sem instalar nada

- ▶ **Playground do perceptron** — arraste pontos no plano, veja a reta $w^T x + b = 0$ se mover a cada atualização da regra; presets E, OU e XOR (para ver a oscilação eterna com os próprios olhos)
- ▶ **O espaço oculto do XOR** — a transformação $(x_1, x_2) \rightarrow (h_1, h_2)$ animada, com controle dos pesos da camada oculta
- ▶ **Aproximador universal 1D** — some “degraus” de neurônios sigmoide para caber numa função-alvo; contador de neurônios visível
- ▶ **Largo vs. fundo** — paridade de n bits: o mesmo problema resolvido raso e fundo, com o número de neurônios lado a lado

Intuição

Regra da disciplina: se um conceito tem geometria, ele ganha um simulador.

Exercício 1 — Portas na mão

Encontre pesos e viés de um perceptron de degrau para NAND. Em seguida, usando só NANDs, desenhe uma rede de duas camadas que computa XOR e escreva os pesos de cada neurônio.

Exercício 2 — O fracasso, observado

Rode o código da regra do perceptron desta aula trocando a porta E por XOR. Registre w e b ao longo de 50 épocas e descreva o que acontece. Depois rode a versão com gradiente e compare.

- ▶ **Leitura:** Goodfellow et al., cap. 1 e seções 6.1–6.4; Nielsen, cap. 1 (aproximação universal, visual)
- ▶ **Opcional:** 11-785 (S26), Lectures 1–2, para o argumento de profundidade vs. largura em detalhe

FUNDADORES

- ▶ McCulloch, W.; Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *Bull. Math. Biophysics* 5.
- ▶ Rosenblatt, F. (1958). The perceptron: a probabilistic model [...]. *Psychological Review* 65(6).
- ▶ Novikoff, A. (1962). On convergence proofs on perceptrons. *Symp. Math. Theory of Automata*.
- ▶ Minsky, M.; Papert, S. (1969). *Perceptrons*. MIT Press.
- ▶ Rumelhart; Hinton; Williams (1986). Learning representations by back-propagating errors. *Nature* 323.
- ▶ Cybenko (1989), MCSS 2; Hornik (1991), *Neural Networks* 4.

Livros: Goodfellow, Bengio & Courville (2016), *Deep Learning*, MIT Press • Nielsen (2015), *neuralnetworksanddeeplearning.com*

Imagens: Wikimedia Commons — E. Bach (CC BY-SA 3.0); National Museum of the U.S. Navy (domínio público).

PROFUNDIDADE E HOJE

- ▶ Håstad (1986). Almost optimal lower bounds for small depth circuits. *STOC*. (tb. Ajtai 1983; Furst, Saxe & Sipser 1984)
- ▶ Telgarsky (2016); Eldan & Shamir (2016). COLT — separações de profundidade para redes contínuas.
- ▶ Krizhevsky; Sutskever; Hinton (2012). ImageNet classification with deep CNNs. *NeurIPS*.
- ▶ Jumper et al. (2021). Highly accurate protein structure prediction with AlphaFold. *Nature* 596.
- ▶ Raj, B. 11-785 *Introduction to Deep Learning*, CMU (S26), Lecs. 1–2 — deeplearning.cs.cmu.edu
- ▶ Amini, A.; Amini, A. *MIT 6.S191: Introduction to Deep Learning* (2026), introtodeeplearning.com — base complementar desta aula.